

Web Application Penetration Testing Report

Organization: CloudExify Summer Internship 2026

Project: Month 2 — Project 3: Web Application Penetration Testing

Assessment Target: Damn Vulnerable Web Application (DVWA v1.10)

Target Host: http://localhost

Primary Tools: Burp Suite Community Edition v2026.7.3, Chromium/Firefox, Linux (Ubuntu)

Assessment Date: August 14, 2026

Auditor / Intern Name: Hammad UL Hassan

1. Executive Summary

This report documents the web application penetration testing assessment conducted on the local deployment of Damn Vulnerable Web Application (DVWA). The objective of this engagement was to identify, exploit, and document security vulnerabilities across the OWASP Top 10 categories, analyze the application attack surface, intercept and manipulate HTTP traffic, and provide production-ready remediation guidance.

The assessment successfully verified vulnerabilities including SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Unrestricted File Upload, and Authentication / Brute Force weaknesses.

2. Assessment Scope & Methodology

The penetration test was executed using a grey-box testing methodology:

- **Reconnaissance & Interception:** Burp Suite Proxy was configured to intercept, inspect, and analyze client-server HTTP transactions.
- **Attack Surface Mapping:** Passive crawling and site mapping were performed via Burp Suite Target mapping.

- **Vulnerability Analysis:** Direct payload injections and security control evaluations were conducted across multiple endpoints.

3. Detailed Vulnerability Findings & Proof of Concept (PoC)

Finding 1: SQL Injection (OWASP A03:2021 — Injection)

- **Endpoint:** /vulnerabilities/sqli/
- **Severity:** Critical
- **Description:** The application dynamically concatenates unvalidated user input directly into the backend SQL query without parameterization.
- **Proof of Concept:**
- **Injected Payload:** ' OR '1'='1
- **Result:** Authentication logic was bypassed, returning the full record set of all user accounts and credentials from the database.
- **Screenshot Evidence:** 03_sqli_dump.png

Finding 2: Reflected Cross-Site Scripting (OWASP A03:2021 — Injection)

- **Endpoint:** /vulnerabilities/xss_r/
- **Severity:** High
- **Description:** User-supplied input is directly echoed back into the HTML document response without proper output escaping or sanitization.
- **Proof of Concept:**

- **Injected Payload:** `<script>alert('CloudExify-XSS')</script>`
- **Result:** Arbitrary JavaScript executed directly within the victim's browser context, triggering a dialog box.
- **Screenshot Evidence:** 04_xss_alert.png

Finding 3: Cross-Site Request Forgery (OWASP A01:2021 — Broken Access Control)

- **Endpoint:** `/vulnerabilities/csrf/`
- **Severity:** High
- **Description:** Sensitive state-changing actions (such as administrative password reset) are transmitted via GET requests without unique Anti-CSRF verification tokens or SameSite cookie protection.
- **Proof of Concept:**
- **Forged Request:** `GET /vulnerabilities/csrf/?password_new=password123&password_conf=password123&Change=Change HTTP/1.1`
- **Result:** An attacker can force authenticated users to execute unauthorized password changes via malicious third-party links or embedded tags.
- **Screenshot Evidence:** 06_csrf_request.png

Finding 4: Unrestricted File Upload (OWASP A04:2021 — Insecure Design)

- **Endpoint:** `/vulnerabilities/upload/`
- **Severity:** Critical

- **Description:** The application accepts arbitrary files without validating file extensions, MIME types, or contents, storing them in a web-accessible directory.
- **Proof of Concept:**
- **Action:** Uploaded an unvalidated script file via the web form.
- **Result:** Server accepted the file with a 200 OK status and returned a direct executable path (../hackable/uploads/).
- **Screenshot Evidence:** 05_file_upload.png

Finding 5: Authentication Weakness & Parameter Fuzzing (OWASP A07:2021 — Identification and Authentication Failures)

- **Endpoint:** /vulnerabilities/brute/
- **Severity:** Medium
- **Description:** The login endpoint lacks rate limiting, CAPTCHA enforcement, and account lockout mechanisms, enabling automated dictionary and credential stuffing attacks.
- **Proof of Concept:**
- **Action:** Intercepted the authentication request and loaded dynamic credential positions into Burp Suite Intruder.
- **Result:** Automated payload deployment allows successful brute-force identification of valid credentials without restriction.
- **Screenshot Evidence:** 07_brute_force_intruder.png

4. Evidence Repository Checklist

Evidence ID	File Name	Assessment Component	Status
PoC 01	01_dwva_setup.png	DVWA Database Initialization & Security Config (Low)	Verified
PoC 02	02_burp_intercept.png	Burp Suite Proxy Request Interception	Verified
PoC 03	03_sql_i_dump.png	SQL Injection Database Dump Execution	Verified
PoC 04	04_xss_alert.png	Reflected XSS JavaScript Alert Execution	Verified
PoC 05	05_file_upload.png	Malicious File Upload Verification	Verified
PoC 06	06_csrf_request.png	Insecure CSRF Request Extraction	Verified
PoC 07	07_brute_force_intruder.png	Burp Intruder Brute Force Payload Configuration	Verified
PoC 08	08_burp_sitemap.png	Complete Target Attack Surface Site Map	Verified

5. Strategic Recommendations

- 1. Implement Parameterized Queries:** Convert all dynamic SQL construction across database access layers into prepared statements.

2. **Context-Aware Output Encoding:** Ensure all user-supplied data reflected in HTML contexts is escaped using `htmlspecialchars()` or automated templating engines.
3. **Anti-CSRF Tokens & Strict Cookies:** Enforce cryptographically strong, per-session anti-CSRF tokens for all state-modifying requests, accompanied by `SameSite=Strict` cookie flags.
4. **Strict File Upload Controls:** Restrict uploads strictly by whitelisting allowable image extensions (`.png`, `.jpg`), enforce server-side MIME verification, and store files outside the document root with non-executable permissions.
5. **Rate Limiting & Account Lockouts:** Implement throttling on authentication endpoints to prevent credential fuzzing and automated dictionary attacks.